

LESSON 8: FILE HANDLING & EXCEPTION HANDLING IN PYTHON

Python Core Engineering Series — In-Depth Study Notes & Architecture Reference

1. FOUNDATIONAL ARCHITECTURE & DATA PERSISTENCE

Computer memory exists across two physical tiers:

1. **Volatile Memory (RAM):** Ultra-fast read/write operations holding active runtime variables, execution stacks, and data structures. It wipes clean the instant the Python process terminates or power is lost.
2. **Non-Volatile Storage (Disk / SSD / NVMe):** Persistent physical blocks retaining data long-term across process recycles, reboots, and deployments.

File handling bridges this boundary, serializing transient data structures into long-term storage formats (.txt, .csv, .json, .bin).

Simultaneously, real-world systems encounter unpredictable runtime conditions: network interruptions, corrupt files, missing directories, invalid user inputs, and unexpected divide-by-zero operations. Exception handling prevents unhandled errors from terminating your application unexpectedly, routing faults into managed fallback routines.

2. FILE HANDLING SUBSYSTEM: MODES, BUFFERS & DESCRIPTORS

When Python interacts with a file, it requests an integer identifier called a File Descriptor from the underlying operating system kernel. The kernel handles the raw block reads, buffering, and physical storage.

Python Script (Application Space)

|

▼ `open("data.txt", "r")`

Standard C Runtime Buffers (I/O Layer)

|

▼ System Calls (read, write, open, close)

Operating System Kernel (File Descriptor Table)

|

▼ Hardware Drivers

Physical Storage Media (SSD / NVMe / HDD)

2.1 File Access Modes Taxonomy

Access Mode	Purpose	File Pointer Starting Position	If File Already Exists	If File Does Not Exist
'r' (Read)	Read-only operations	Beginning (offset 0)	Reads existing content	Raises <code>FileNotFoundError</code>
'w' (Write)	Overwrite / Create	Beginning (offset 0)	Truncates (erases) file!	Creates a new file
'a' (Append)	Append / Create	End of file (EOF)	Preserves old data; appends	Creates a new file
'x' (Exclusive)	Strict creation	Beginning (offset 0)	Raises <code>FileExistsError</code>	Creates a new file
'r+' (Read/Write)	In-place stream mutation	Beginning (offset 0)	Modifies content in place	Raises <code>FileNotFoundError</code>
'w+' (Write/Read)	Overwrite & Read	Beginning (offset 0)	Truncates existing file!	Creates a new file
'a+' (Append/Read)	Append & Read	End of file (EOF)	Appends; reads on seek	Creates a new file

- **Text vs. Binary Modifier:** Suffixing 'b' (e.g., 'rb', 'wb') opens the file in raw binary mode (returning bytes), which is mandatory for audio, images, PDFs, and compiled objects. Suffixing 't' (or omitting it) operates in text mode, automatically encoding and decoding Unicode strings.

3. READING & WRITING PARADIGMS: MEMORY EFFICIENCY

Reading data can be handled in several ways, each with distinct memory trade-offs:

3.1 The Three Reading Primitives

- `f.read()`: Loads the entire file contents into a single string in RAM.
 - *Hazard*: Reading a 16 GB database export on a machine with 8 GB of RAM causes memory exhaustion and crashes the process.
- `f.readlines()`: Reads the entire file and returns a list of individual line strings.
 - *Hazard*: Still loads the entire file into memory simultaneously.
- Line-by-Line Generator Iteration (for line in f):
 - *The Production Standard*: Streams lines one by one from kernel buffers using Python's generator protocol. Memory consumption remains low regardless of file size ($O(1)$ memory usage).

Python

❌ DANGEROUS FOR LARGE FILES: Loads 20 GB into RAM at once

with open("massive_dataset.csv", "r") as f:

```
records = f.read()
```

✅ PRODUCTION PATTERN: Constant $O(1)$ memory streaming

with open("massive_dataset.csv", "r") as f:

```
for line in f:
```

```
    process_line(line.strip())
```

4. THE CONTEXT MANAGER PROTOCOL (with open())

In legacy code, working with files required an explicit `.close()` call:

Python

```
f = open("data.txt", "r")
```

```
data = f.read()
```

```
f.close()
```

If `f.read()` raised an unexpected exception, execution skipped `f.close()`. The file descriptor remained locked in the operating system kernel, causing resource leaks.

How with open(...) Solves This

The `with` statement utilizes the Context Manager Protocol, binding execution to two magic methods:

1. `__enter__()`: Executes before entering the indented block, allocating the resource and binding it to the identifier (e.g., as `f`).
2. `__exit__()`: Guaranteed to execute when the block exits, whether execution finishes normally, uses `return`, or raises an unhandled runtime exception. It automatically flushes buffers and closes the kernel file descriptor.

Python

```
with open("audit.log", "a", encoding="utf-8") as file_handle:
```

```
    file_handle.write("System online.\n")
```

```
# File handle is cleanly closed here automatically
```

5. EXCEPTION HANDLING: THE PYTHON ERROR HIERARCHY

An Exception is an event that breaks the normal instruction flow when an error occurs during runtime.

5.1 The Built-in Exception Hierarchy (Subset)

In Python, all exceptions inherit from `BaseException`:

`BaseException`

- └─ `SystemExit`

- └─ `KeyboardInterrupt`

- └─ `Exception` (Standard User & Library Hierarchy)

 - └─ `ArithmeticError`

 - └─ `ZeroDivisionError`

 - └─ `OverflowError`

 - └─ `LookupError`

 - └─ `IndexError`

- | └─ **KeyError**
- | └─ **ValueError**
- | └─ **TypeError**
- | └─ **FileNotFoundError** (subclass of **OSError**)
- | └─ **PermissionError** (subclass of **OSError**)

5.2 Why You Should Never Use a Bare except:

Writing `except:` without an exception class catches `BaseException`, including system interrupts:

- It intercepts `KeyboardInterrupt` (preventing users from stopping the program with Ctrl + C).
- It intercepts `SystemExit` (breaking clean application shutdowns).
- Always catch specific exceptions (e.g., `except ValueError:`) or catch `except Exception:` if a general catch-all is necessary.

6. THE COMPLETE CONTROL FLOW: try-except-else-finally

To handle errors cleanly, Python provides a unified four-part exception-handling block:

try → [Fault Detected?] ^{Yes} → except | ^{No} → else → finally (Always runs)

Python

try:

1. Monitored Code Suite: Operations that might trigger a runtime fault

```
raw_input_data = "42"
```

```
parsed_value = int(raw_input_data)
```

```
division_result = 100 / parsed_value
```

except ValueError as val_err:

2. Targeted Handler: Executes ONLY if a ValueError is raised

```
print(f"Data conversion fault: {val_err}")
```

```
except ZeroDivisionError as zero_err:
```

```
# 3. Targeted Handler: Executes ONLY if division by zero occurs
```

```
print(f"Math calculation fault: {zero_err}")
```

```
else:
```

```
# 4. Success Suite: Executes ONLY if the try block completed with ZERO errors
```

```
print(f"Calculation completed smoothly. Result: {division_result}")
```

```
finally:
```

```
# 5. Cleanup Suite: Executes unconditionally under all circumstances
```

```
# Runs after try, after except, or even if an unhandled error occurred
```

```
print("Cleanup cycle complete.")
```

7. EXPLICIT RAISING & CUSTOM DOMAIN EXCEPTIONS

7.1 The raise Statement

Use raise to stop execution intentionally when an invalid state, boundary violation, or invariant failure occurs:

Python

```
def configure_worker_pool(thread_count: int):
```

```
    if not isinstance(thread_count, int):
```

```
        raise TypeError(f"Worker count must be an integer, got:  
{type(thread_count).__name__}")
```

```
    if thread_count <= 0 or thread_count > 64:
```

```
        raise ValueError(f"Worker pool must be between 1 and 64 threads. Received:  
{thread_count}")
```

```
    return f"Allocated {thread_count} worker threads."
```

7.2 Building Domain-Specific Custom Exceptions

In enterprise applications, generic exceptions like `ValueError` can be ambiguous. Creating custom domain exceptions that inherit from `Exception` makes error handling explicit and easy to track across modules:

Python

```
class BankingException(Exception):
```

```
    """Base domain exception for all financial operations."""
```

```
    pass
```

```
class InsufficientFundsError(BankingException):
```

```
    """Raised when an account lacks sufficient balance for a withdrawal."""
```

```
    def __init__(self, requested: float, available: float):
```

```
        super().__init__(
```

```
            f'Insufficient funds: Attempted to withdraw ${requested:.2f}, "
```

```
            f"but current balance is only ${available:.2f}."
```

```
        )
```

```
        self.requested = requested
```

```
        self.available = available
```

8. COMPLETE PRODUCTION CODE SCRIPT

The following standalone script demonstrates file writes, reads, memory-efficient iterations, error-handling flows, explicit validation, and custom domain exceptions:

Python

```
"""
```

Lesson 8: File I/O Subsystems & Defensive Exception Handling

A complete demonstration of file read/write operations, streaming iterators, try-except-else-finally blocks, input validation, and custom domain exceptions.

```
"""
```

```
import os
```

```
# =====
```

```
# 1. FILE SYSTEM OPERATIONS: WRITE, APPEND, STREAMING ITERATION
```

```
# =====
```

```
print("--- 1. File Writing and Streaming Operations ---")
```

```
LOG_FILENAME = "telemetry_records.txt"
```

```
# Safe File Write using Context Manager (Truncates & Writes)
```

```
with open(LOG_FILENAME, "w", encoding="utf-8") as file_out:
```

```
    file_out.write("2026-09-04 10:00:01|SYS_INIT|Core system boot sequence\n")
```

```
    file_out.write("2026-09-04 10:00:05|NETWORK_UP|Interface eth0 linked 10Gbps\n")
```

```
    file_out.write("2026-09-04 10:00:12|AUTH_FAIL|Invalid credentials for user: unknown\n")
```

```
print(f"File successfully created: '{LOG_FILENAME}'")
```

```
# Safe File Append Mode
```

```
with open(LOG_FILENAME, "a", encoding="utf-8") as file_out:
```

```
    file_out.write("2026-09-04 10:00:19|AUTH_FAIL|Failed password attempt: user admin\n")
```

```
    file_out.write("2026-09-04 10:01:00|SHUTDOWN_SIG|Termination signal received\n")
```

```
# Reading via Memory-Efficient Line-by-Line Iteration
```

```
print(f"\nProcessing Log Records from '{LOG_FILENAME}':")
```

```
with open(LOG_FILENAME, "r", encoding="utf-8") as file_in:
```



```

for line_number, raw_record in enumerate(file_in, start=1):

    clean_record = raw_record.strip()

    timestamp, event_type, details = clean_record.split(" | ")

    print(f" Line {line_number} [{timestamp}] Level: {event_type:<12} -> {details}")

# =====

# 2. EXCEPTION HANDLING ARCHITECTURE: TRY-EXCEPT-ELSE-FINALLY

# =====

print("\n--- 2. Comprehensive Try-Except-Else-Finally Pipeline ---")


def execute_numerical_division(numerator_str: str, denominator_str: str):
    """
    Parses string inputs and executes division within a guarded sandbox.
    """
    print(f"\nExecuting Division: {numerator_str} / {denominator_str}")
    try:
        n = float(numerator_str)
        d = float(denominator_str)
        result = n / d
    except ValueError as val_err:
        print(f" [EXCEPT] Conversion Error: Non-numeric input provided ({val_err})")
    except ZeroDivisionError as zero_err:
        print(f" [EXCEPT] Mathematical Fault: Cannot divide by zero ({zero_err})")
    else:
        print(f" [ELSE] Calculation successful. Result: {result:.4f}")
    finally:

```

```

    print(" [FINALLY] Releasing computational sandbox resources.")

# Exercise the four pathways

execute_numerical_division("100", "4") # Hits try -> else -> finally
execute_numerical_division("100", "0") # Hits try -> except ZeroDivisionError -> finally
execute_numerical_division("one", "5") # Hits try -> except ValueError -> finally

# =====

# 3. DEFENSIVE FILE ACCESS & OPERATING SYSTEM CHECK

# =====

print("\n--- 3. Defensive File Access & FileNotFoundError ---")

NON_EXISTENT_FILE = "ghost_metrics_data.csv"

try:
    with open(NON_EXISTENT_FILE, "r", encoding="utf-8") as file_handle:
        data = file_handle.read()
except FileNotFoundError as err:
    print(f"Intercepted missing file error cleanly: {err.filename} is not on disk.")

# Check file presence using the os module

if os.path.exists(LOG_FILENAME):
    print(f"Verification: '{LOG_FILENAME}' exists. File size: {os.path.getsize(LOG_FILENAME)} bytes.")

# =====

```

4. CUSTOM DOMAIN EXCEPTIONS & INPUT VALIDATION

```
# =====
```

```
print("\n--- 4. Custom Enterprise Domain Exceptions ---")
```

```
class AccountSecurityException(Exception):
```

```
    """Base exception for user account authorization failures."""
```

```
    pass
```

```
class AccountSuspendedError(AccountSecurityException):
```

```
    """Raised when an operation is attempted on an inactive account."""
```

```
    def __init__(self, username: str, reason: str):
```

```
        super().__init__(f"Account for '{username}' is suspended. Reason: {reason}")
```

```
        self.username = username
```

```
        self.reason = reason
```

```
def authenticate_bank_customer(username: str, is_active: bool):
```

```
    if not is_active:
```

```
        raise AccountSuspendedError(username, "Repeated fraudulent access attempts  
detected")
```

```
    return f"Access granted to profile: {username}"
```

```
try:
```

```
    print(authenticate_bank_customer("alex_dev", is_active=True))
```

```
    print(authenticate_bank_customer("bad_actor", is_active=False))
```

```
except AccountSuspendedError as sec_err:
```

```
    print(f"Security Alert Triggered: {sec_err}")
```

```
print(f"Flagged Username: {sec_err.username} | Case: {sec_err.reason}")
```

```
# =====
```

```
# 5. CLEANUP ROUTINE
```

```
# =====
```

```
# Clean up the generated file to leave the workspace tidy
```

```
if os.path.exists(LOG_FILENAME):
```

```
    os.remove(LOG_FILENAME)
```

```
    print(f"\nCleanup complete: '{LOG_FILENAME}' removed from local disk.")
```

9. STEP-BY-STEP CODE FLOW & EXECUTION TRACE

1. Context Manager Allocation (with open(...) as file_out:):

Python enters the block, creates a file named `telemetry_records.txt` in write mode ('w'), and assigns the file descriptor to `file_out`. If the file previously existed, its contents would be truncated immediately.

2. Deterministic Append Mode ('a'):

Re-opening the file with mode 'a' moves the internal file pointer directly to the End of File (EOF), appending new lines without modifying prior records.

3. Low-Memory Streaming Loop (for line in file_in:):

Instead of reading the entire file into memory at once with `f.read()`, the loop yields lines one at a time via internal buffers. Each line is stripped of trailing newlines and parsed using `.split("|")`.

4. The try-except-else-finally Lifecycle:

- When given "100" and "4", the try block succeeds completely. The except blocks are skipped, the else block runs to display the result, and finally runs to clean up resources.
- When given "100" and "0", the division triggers a `ZeroDivisionError`. Python immediately jumps to the matching except `ZeroDivisionError:` block. The else block is skipped, and finally runs immediately afterward.

5. Custom Exception Propagation:

When `authenticate_bank_customer()` encounters `is_active=False`, it creates an `AccountSuspendedError` instance and raises it. The calling scope intercepts it with `except AccountSuspendedError:` and inspects custom attributes (`username`, `reason`) attached to the error object.

10. JUNIOR ANTI-PATTERNS & PRODUCTION PITFALLS

Anti-Pattern 1: Catching and Silencing Exceptions Blindly

Python

❌ ANTI-PATTERN: Completely silences bugs; makes debugging impossible

try:

```
    process_transactions()
```

except Exception:

```
    pass # Swallows the error silently!
```

✅ CORRECT: Log the error details explicitly or handle specific exceptions

try:

```
    process_transactions()
```

except DatabaseConnectionError as err:

```
    logger.error("Failed to connect to primary DB: %s", err)
```

```
    raise
```

Anti-Pattern 2: Manually Opening Files Without a Context Manager

Python

❌ FRAGILE: If an error occurs during parsing, `f.close()` is never reached

```
f = open("transactions.csv", "r")
```

```
data = f.read()
```

```
parse(data) # If this raises an exception, the file descriptor remains open!
```

```
f.close()
```

 **CORRECT:** The context manager guarantees clean closure under all conditions

with open("transactions.csv", "r") as f:

```
    data = f.read()
```

```
    parse(data)
```

Anti-Pattern 3: Opening Files Without Specifying Character Encoding

Python

 **INCONSISTENT:** Uses the OS default encoding (e.g., Windows-1252 vs UTF-8)

with open("data.txt", "r") as f:

```
    ...
```

 **CORRECT:** Always declare UTF-8 explicitly to avoid cross-platform encoding bugs

with open("data.txt", "r", encoding="utf-8") as f:

```
    ...
```

11. TECHNICAL INTERVIEW & RAPID RECAP CHEAT SHEET

High-Yield Questions & Answers

Q1: How does a Python context manager (with statement) work under the hood?

Answer: A context manager implements the runtime context methods `__enter__()` and `__exit__()`. When entering the with block, `__enter__()` executes and returns the target resource. When exiting the block, `__exit__(exc_type, exc_val, exc_tb)` executes automatically. If an exception occurred inside the block, its type, value, and traceback are passed into `__exit__()`, allowing the manager to clean up resources before either swallowing or propagating the error.

Q2: What is the exact execution timing of the else clause in a try-except structure?

Answer: The else block runs only if the code inside the try block completes without raising any exceptions. It is used for code that should run only on success, keeping the try block small and focused strictly on the lines that might fail.

Q3: What is the difference between `f.read()`, `f.readline()`, and `f.readlines()`?

Answer: `f.read()` loads the entire file contents into a single string in memory. `f.readline()` reads a single line up to the next newline character (`\n`). `f.readlines()` reads all remaining lines into a list of strings. For large files, iterating directly over the file handle (`for line in f:`) is preferred to stream lines with minimal memory usage.

Q4: What is the difference between `Exception` and `BaseException` in Python?

Answer: `BaseException` is the root class of all exceptions in Python. It has subclasses like `KeyboardInterrupt`, `SystemExit`, and `GeneratorExit`. `Exception` inherits from `BaseException` and serves as the base class for standard application-level errors (`ValueError`, `KeyError`, `OSError`). Application code should inherit from and catch `Exception`, not `BaseException`, to allow normal system interrupts to function.